

CSE 2201: Database Management Systems

Lecture 12: Functional Dependencies – Clear Examples and Design Reasoning

Md Mynoddin
Assistant Professor (CSE), RMSTU

Department of Computer Science and Engineering

Lecture Outcomes

After this lecture, students should be able to:

- 1 explain why redundancy creates update, insertion and deletion anomalies;
- 2 define functional dependency using plain language and notation;
- 3 distinguish trivial, nontrivial and completely nontrivial FDs;
- 4 compute attribute closure step by step;
- 5 identify superkeys and candidate keys from FDs;
- 6 apply Armstrong's axioms and useful derived rules;
- 7 test whether a decomposition is lossless;
- 8 build a canonical cover for a small FD set.

Main Theme

Functional dependencies are the mathematical language for expressing design rules in relational databases.

From ER Design to Normalization

Stage	Main Question
ER model	What entities and relationships exist?
Relational schema	What tables and attributes should be created?
Functional dependencies	Which attributes determine other attributes?
Normalization	How can we reduce redundancy without losing information?

Important

A relation may satisfy SQL syntax but still be a poor design if it violates real-world dependencies.

Motivating Example: One Large Table

InstDept(ID, name, salary, dept_name, building, budget)

ID	name	salary	dept_name	building	budget
101	Rahman	95000	CSE	A-Building	5000000
102	Akter	65000	CSE	A-Building	5000000
201	Karim	92000	EEE	B-Building	4200000
301	Sen	88000	Math	C-Building	2600000

Hidden Rules

$ID \rightarrow name, salary, dept_name$

$dept_name \rightarrow building, budget$

What Goes Wrong? Three Anomalies

Update anomaly

If the CSE budget changes, every CSE instructor row must be updated.

Insertion anomaly

A new department cannot be stored unless an instructor already exists.

Deletion anomaly

Deleting the last instructor of a department may delete the department facts.

Better Idea

Separate instructor facts from department facts:

```
Instructor(ID, name, salary, dept_name)
```

```
Department(dept_name, building, budget)
```

Functional Dependency: Plain Meaning

Definition

For a relation schema R , a functional dependency

$$\alpha \rightarrow \beta$$

holds on R if whenever two tuples have the same values for all attributes in α , they must also have the same values for all attributes in β .

Plain English

`student_id` $\rightarrow$ `student_name`, `dept_id`

means: if two rows have the same student ID, they must refer to the same student name and department.

Functional Dependency Example with Data

Relation:

Student(student_id, student_name, email, dept_id, cgpa)

student_id	student_name	email	dept_id	cgpa
2201	Nowshin	nowshin@example.com	1	3.86
2202	Avijit	avijit@example.com	1	3.54
2401	Arif	arif@example.com	2	3.67

Likely FDs

$\text{student_id} \rightarrow \text{student_name, email, dept_id, cgpa}$

$\text{email} \rightarrow \text{student_id, student_name}$

Not a safe FD

$\text{dept_id} \nrightarrow \text{student_name}$

A department can have many students.

FDs Are Business Rules, Not Accidental Data Patterns

Common Mistake

Do not decide an FD only from the few rows currently stored in a table.

Example

If today's table has only one student from each city, we still should not conclude:

$$\text{city} \rightarrow \text{student_id}$$

because the business rule allows many students from the same city.

Correct Thinking

An FD must hold for every legal instance of the relation, not just the current sample data.

Trivial, Nontrivial and Completely Nontrivial FDs

Type	Rule	Example
Trivial	$\beta \subseteq \alpha$	$\{ID, name\} \rightarrow ID$
Nontrivial	Some attributes in β are not in α	$ID \rightarrow name$
Completely nontrivial	$\alpha \cap \beta = \emptyset$	$dept_name \rightarrow budget$

Why trivial FDs matter

They are always true, but they usually do not reveal useful design rules.

Keys and Functional Dependencies

Superkey

A set of attributes K is a superkey for relation R if:

$$K \rightarrow R$$

That means K determines all attributes of R .

Candidate Key

A candidate key is a minimal superkey: no proper subset of it is still a superkey.

Example

For Student(student_id, email, name, dept):

$\text{student_id} \rightarrow \text{email, name, dept}$

$\text{email} \rightarrow \text{student_id, name, dept}$

Both student_id and email may be candidate keys.

Attribute Closure: The Main Tool

Definition

The closure of a set of attributes α , written α^+ , is the set of all attributes functionally determined by α using a given FD set F .

Why calculate closure?

Attribute closure helps us answer:

- Is α a superkey?
- Does $\alpha \rightarrow \beta$ follow from F ?
- Which attributes can be inferred from a known attribute set?

Attribute Closure Algorithm

Algorithm for α^+

- 1 Start with $result = \alpha$.
- 2 For every FD $X \rightarrow Y$ in F , if $X \subseteq result$, then add Y to $result$.
- 3 Repeat step 2 until no more attributes can be added.
- 4 The final $result$ is α^+ .

Practical Tip

Always write the closure as a growing set. It prevents mistakes in exams.

Worked Example 1: Compute Closure

Let

$$R(A, B, C, D, E), \quad F = \{A \rightarrow B, B \rightarrow C, CD \rightarrow E, A \rightarrow D\}$$

Find A^+ .

Step	Reason	Result
Start	begin with A	$\{A\}$
1	$A \rightarrow B$	$\{A, B\}$
2	$B \rightarrow C$	$\{A, B, C\}$
3	$A \rightarrow D$	$\{A, B, C, D\}$
4	$CD \rightarrow E$ because C, D available	$\{A, B, C, D, E\}$

Conclusion

$A^+ = ABCDE$, so A is a superkey for R .

Worked Example 2: Is an FD Implied?

Given:

$$R(A, B, C, D), \quad F = \{A \rightarrow B, B \rightarrow C, C \rightarrow D\}$$

Question: Does $A \rightarrow D$ follow from F ?

Use closure

Compute A^+ :

$$\{A\} \Rightarrow \{A, B\} \Rightarrow \{A, B, C\} \Rightarrow \{A, B, C, D\}$$

Answer

Yes. Since $D \in A^+$, the dependency $A \rightarrow D$ is implied by F .

Finding Candidate Keys: Example

Let

$$R(A, B, C, D), \quad F = \{A \rightarrow B, B \rightarrow C\}$$

Step 1: Attributes never appearing on RHS

RHS attributes are B and C . Attributes not on RHS: A, D .

Step 2: They must be in every key

Try AD :

$$AD^+ = \{A, D\} \Rightarrow \{A, B, D\} \Rightarrow \{A, B, C, D\}$$

Conclusion

AD is a candidate key. Neither A nor D alone determines all attributes.

Armstrong's Axioms

Axiom	Rule	Meaning
Reflexivity	If $\beta \subseteq \alpha$, then $\alpha \rightarrow \beta$	A set determines its subset.
Augmentation	If $\alpha \rightarrow \beta$, then $\gamma\alpha \rightarrow \gamma\beta$	Add same attributes to both sides.
Transitivity	If $\alpha \rightarrow \beta$ and $\beta \rightarrow \gamma$, then $\alpha \rightarrow \gamma$	Dependency chain.

Important

These rules are sound and complete for reasoning about functional dependencies.

Useful Derived Rules

Rule	Form	Example
Union	If $A \rightarrow B$ and $A \rightarrow C$, then $A \rightarrow BC$	$ID \rightarrow name$ and $ID \rightarrow dept$ imply $ID \rightarrow name, dept$
Decomposition	If $A \rightarrow BC$, then $A \rightarrow B$ and $A \rightarrow C$	$ID \rightarrow name, dept$ implies $ID \rightarrow name$
Pseudotransitivity	If $A \rightarrow B$ and $CB \rightarrow D$, then $AC \rightarrow D$	useful in composite dependencies

Lossless Decomposition Test for Two Relations

For decomposition of R into R_1 and R_2 :

Lossless Condition

The decomposition is lossless if at least one holds:

$$(R_1 \cap R_2) \rightarrow R_1$$

or

$$(R_1 \cap R_2) \rightarrow R_2$$

Intuition

The common attributes must be able to identify tuples in at least one of the decomposed relations.

Worked Example: Lossless or Lossy?

Relation:

$$R(A, B, C), \quad F = \{A \rightarrow B\}$$

Decomposition:

$$R_1(A, B), \quad R_2(A, C)$$

Test

$$R_1 \cap R_2 = \{A\}$$

Since $A \rightarrow B$, we have:

$$A \rightarrow AB = R_1$$

Conclusion

The decomposition is lossless.

Dependency Preservation: Why It Matters

Definition

A decomposition is dependency-preserving if the original FDs can be checked by enforcing dependencies on individual decomposed relations, without joining them back.

Why care?

If dependencies are not preserved, the database may need expensive joins merely to check constraints.

Design Goal

Good decomposition should ideally be:

- lossless, and
- dependency-preserving.

Extraneous Attribute: Intuition

Extraneous Attribute

An attribute in an FD is extraneous if it can be removed without changing the closure of the FD set.

Example

If

$$F = \{A \rightarrow B, AB \rightarrow C\}$$

then B is extraneous in $AB \rightarrow C$ because A already gives B , so A can determine C through $AB \rightarrow C$.

Purpose

Removing extraneous attributes helps create a simpler canonical cover.

Canonical Cover: Clean FD Set

Canonical Cover

A canonical cover F_c is a simplified FD set equivalent to F , with:

- no extraneous attributes,
- each left side appearing only once where possible,
- simpler dependencies for normalization algorithms.

Typical Procedure

- 1 Decompose RHS so each FD has one attribute on RHS.
- 2 Remove extraneous attributes from LHS and RHS.
- 3 Combine FDs with the same LHS.

Worked Canonical Cover Example

Let

$$F = \{A \rightarrow BC, B \rightarrow C, A \rightarrow B, AB \rightarrow C\}$$

- 1 Decompose RHS: $A \rightarrow B, A \rightarrow C, B \rightarrow C, A \rightarrow B, AB \rightarrow C$.
- 2 $A \rightarrow C$ is implied by $A \rightarrow B$ and $B \rightarrow C$, so remove it.
- 3 In $AB \rightarrow C$, A is extraneous because $B \rightarrow C$ already holds.
- 4 Duplicate dependencies can be removed.

One canonical cover

$$F_c = \{A \rightarrow B, B \rightarrow C\}$$

SQL Connection: Constraints and FDs

```
CREATE TABLE student (  
  student_id INT PRIMARY KEY,  
  email VARCHAR(120) UNIQUE,  
  student_name VARCHAR(80) NOT NULL,  
  dept_id INT NOT NULL  
);
```

FDs implied by constraints

$\text{student_id} \rightarrow \text{email}, \text{student_name}, \text{dept_id}$

$\text{email} \rightarrow \text{student_id}, \text{student_name}, \text{dept_id}$

But

Many business FDs are not automatically enforced unless we design keys and constraints properly.

In-Class Practice

For each relation, list likely FDs and candidate keys.

- 1 Course(course_id, title, credits, dept_id)
- 2 Enrollment(student_id, section_id, marks, grade)
- 3 Room(room_no, building, capacity, room_type)
- 4 OrderItem(order_id, product_id, quantity, unit_price)

Challenge

For $R(A, B, C, D, E)$ and $F = \{AB \rightarrow C, C \rightarrow D, D \rightarrow E\}$, compute AB^+ and decide whether AB is a superkey.

Lecture Summary

- Functional dependencies express semantic constraints between attributes.
- Redundancy and anomalies often reveal hidden dependencies.
- Attribute closure is the main practical tool for FD reasoning.
- Keys can be identified using closure.
- Armstrong's axioms justify FD inference.
- A good decomposition should be lossless and preferably dependency-preserving.
- Canonical cover simplifies FD sets before normalization.

Next Lecture

We will use FDs to normalize relations into 1NF, 2NF and 3NF.